SOFTWARE REQUIREMENTS SPECIFICATION

Client E-Commerce Application

Badminton Equipment & Accessories Store

Suggestive Selling + Voucher at Checkout

Platform: MedusaJS v2

Version 1.0 | July 2026

CONFIDENTIAL — Internal Use Only

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for two interconnected features of the Client E-Commerce Application (a badminton equipment store built on MedusaJS v2): **Suggestive Selling** and **Voucher** at Checkout. These two features are tightly coupled — suggestive selling drives additional items into the cart, and the voucher system must correctly handle discount calculations when both regular items and suggested items coexist, including conflict resolution and discount capping.

This document serves as the single source of truth for implementation, testing, and acceptance validation of these features.

1.2 Scope

In Scope	Out of Scope
Suggestive selling rules (product-level and cart-level)	Product catalog management and menu browsing
Suggestion display, interaction, and analytics tracking	User authentication and account management
Voucher validation, application, and removal at checkout	Payment processing (MoMo, Payoo, COD)
Discount calculation with stacking rules and cap enforcement	Order tracking and delivery
Conflict resolution: suggestion discounts vs voucher discounts	Store locator and address management
MedusaJS Module/Workflow/Subscriber implementation patterns	CMS and admin panel for rule management (API only)

1.3 Definitions

Term	Definition
Suggestive Selling	Recommending complementary or higher-value products based on what the user is viewing or has in cart, to increase average order value
Product-Level Suggestion	Recommendations shown on a product detail page (e.g., viewing a racket → suggest matching string, grip, bag)
Cart-Level Suggestion	Recommendations shown on the cart page based on aggregate cart contents (e.g., cart has racket but no string → suggest strings)
Voucher	A digital discount code applied at checkout with configurable validation rules (min order, scope, expiry, usage limit)
Discount Cap	Maximum total discount (as percentage of cart subtotal) allowed from all combined sources (item promotions + voucher)
Stacking	Whether a voucher can be combined with other active discounts (item-level promotions, other vouchers)
MedusaJS Module	Self-contained domain unit with its own data models, services, and API routes
MedusaJS Workflow	Orchestration primitive for multi-step, compensatable business operations

Term	Definition
MedusaJS Subscriber	Event-driven handler that reacts to domain events (e.g., cart.updated)
Link Module	MedusaJS mechanism for creating cross-module entity relationships without tight coupling

2. System Context

2.1 Architecture Overview

Both features are implemented as custom MedusaJS v2 modules that extend the existing commerce engine. They interact with MedusaJS built-in modules (Product, Cart, Promotion, Pricing) through the Link Module pattern and event-driven subscribers — no direct database coupling between modules.

Component	Type	Interacts With
SuggestiveSelling Module	Custom Module (new)	Product Module (read products), Cart Module (read cart items), Link Module (product-to-suggestion mapping), Redis (cache)
VoucherEngine Module	Custom Module (extends Promotion)	Promotion Module (base discount logic), Cart Module (apply discount to cart), SuggestiveSelling Module (check if suggested items have own discounts)
Cart Module (built-in)	Extended	Receives events from both custom modules. Price recalculation triggered on voucher apply/remove and suggestion add/remove
Redis	Infrastructure	Caches suggestion results (TTL 5min), voucher validation results (TTL 30sec), stock availability for suggested products

2.2 User Stories

US-01: As a customer viewing a racket, I want to see complementary products (strings, grips, bags) so I can complete my setup in one purchase.

US-02: As a customer on the cart page, I want to see smart recommendations based on what's in my cart so I discover items I might have missed.

US-03: As a customer at checkout, I want to apply a voucher code and immediately see the updated total so I know exactly how much I'm saving.

US-04: As a customer, I want the system to handle discount conflicts automatically so I always get the best valid deal without manual calculation.

US-05: As an admin, I want to configure suggestion rules and voucher parameters via API so I can run targeted promotions without code changes.

3. Functional Requirements: Suggestive Selling

3.1 Product-Level Suggestions

[SUGG-001] Complementary Product Recommendations — Must Have

On the product detail page, the system shall display a 'Complete Your Setup' section showing 3-5 complementary products. The suggestion engine evaluates rules in a tiered priority order:

Tier 1 — Manual Curation: Admin-configured product-to-product links via API (e.g., Racket X → [String A, Grip B, Bag C]). Highest priority, always displayed first.

Tier 2 — Category Complement: If Tier 1 yields fewer than 3 results, fill remaining slots with top-selling products from complementary categories. Category complement mapping: Rackets → Strings, Grips, Bags; Shoes → Socks, Insoles; Shuttlecocks → Tubes (bulk).

Tier 3 — Behavioral (Phase 2): Based on purchase history and browsing patterns. Out of scope for initial release but data model supports it.

Acceptance Criteria:

GIVEN admin has configured: 'Yonex Astrox 99 Pro' → suggests ['BG65 String', 'Astrox Racket Bag', 'Yonex Cushion Grip']

WHEN a customer views the 'Yonex Astrox 99 Pro' detail page

THEN the 'Complete Your Setup' section displays all 3 products with image, name, price, and one-tap 'Add to Cart' button

AND products are ordered by the display_order configured by admin

GIVEN admin has configured only 1 manual suggestion for 'Li-Ning Axforce 80'

WHEN a customer views the product

THEN 1 manual suggestion is shown first, followed by 2-4 category-complement products to fill the section

[SUGG-002] Suggestion Filtering Rules — Must Have

A suggested product shall be excluded from display if ANY of the following is true: (a) the product is already in the customer's cart, (b) the product is out of stock at the customer's assigned store, (c) the customer has dismissed this specific suggestion in the current session (tap 'X' or swipe away), (d) the product has been purchased by this customer within the last 30 days.

Acceptance Criteria:

GIVEN 'BG65 String' is a configured suggestion for 'Astrox 99 Pro'

AND the customer already has 'BG65 String' in their cart

WHEN the customer views 'Astrox 99 Pro'

THEN 'BG65 String' is NOT shown in suggestions

AND the slot is filled by the next eligible product from Tier 1 or Tier 2

[SUGG-003] One-Tap Add from Suggestion — Must Have

Each suggested product shall have an 'Add to Cart' button that adds the default variant (or the only variant) to the cart with quantity 1. If the product has multiple variants with no default, tapping 'Add' opens a compact variant selector (bottom sheet on mobile) instead of navigating to the full product detail page. After adding, the suggestion card updates to show a checkmark with 'Added' state for 3 seconds, then returns to normal state. A toast notification confirms: '{Product Name} added to cart' with a 3-second 'Undo' action.

3.2 Cart-Level Suggestions

[SUGG-004] Cart-Based Cross-Sell Recommendations — Must Have

On the cart page, the system shall display a 'You Might Also Need' section with up to 3 product suggestions. Cart-level rules analyze the aggregate cart contents:

Rule ID	Condition	Action	Example
CR-01	Cart contains category X but not complementary category Y	Suggest top-selling from category Y	Has racket, no string → suggest strings
CR-02	Cart total is within 15% of a promotional threshold	Suggest affordable items that push cart over threshold	Cart 590K, free shipping at 700K → suggest items under 150K with badge 'Add for FREE shipping!'
CR-03	All cart items are same brand	Suggest same-brand accessories	All Yonex items → suggest Yonex accessories
CR-04	Cart has consumable with quantity 1	Suggest bulk/multipack of same item	1 tube shuttlecocks → suggest 3-tube bundle at better per-unit price

Rules are evaluated in priority order (CR-01 → CR-04). If multiple rules fire, take the top 3 unique suggestions across all rules. If fewer than 3, show what's available (never show an empty section — hide the entire section if 0 suggestions).

Acceptance Criteria:

GIVEN cart contains: 'Yonex Astrox 99 Pro' (racket, 4,500,000 VND) and 'Yonex 65Z Shoes' (2,200,000 VND)

AND free shipping threshold is 7,000,000 VND (cart total = 6,700,000, within 15%)

AND cart has racket but no string (CR-01 fires) AND threshold is near (CR-02 fires)

WHEN the cart page renders

THEN suggestions show: (1) top-selling string [from CR-01], (2) affordable item under 400K with 'Add for FREE shipping!' badge [from CR-02], (3) next best CR-01 or CR-03 result

[SUGG-005] Suggestion Refresh on Cart Change — Must Have

Suggestions shall be re-evaluated whenever the cart contents change (item added, removed, or quantity updated). The re-evaluation is triggered by a cart.updated subscriber and results are cached in Redis with a key pattern cart:{cart_id}:suggestions and a TTL of 5 minutes. Cache is invalidated immediately on cart change (not waiting for TTL).

[SUGG-006] Suggestion Analytics — Should Have

Every suggestion interaction shall be tracked: impression (suggestion rendered on screen), tap (user tapped the product card), add_to_cart (user added the suggested product), dismiss (user closed/swiped the suggestion). Each event records: suggestion_rule_id, source_context (product_view | cart), source_product_id (for product-level) or 'cart' (for cart-level), suggested_product_id, customer_id, session_id, timestamp, action.

4. Functional Requirements: Voucher at Checkout

4.1 Voucher Application

[VOUCH-001] Apply Voucher Code — Must Have

At the checkout step, the customer can apply a voucher via: (a) manual code entry — text input with 'Apply' button, or (b) selection from 'My Vouchers' list — vouchers assigned to the customer's account via CRM campaigns. Only one voucher can be active at a time. Applying a new voucher replaces the existing one (with confirmation: 'Replace current voucher {code}?').

Acceptance Criteria:

GIVEN customer has voucher 'SHUTTLE20' (20% off Shuttlecocks, min order 200,000 VND, max discount 100,000 VND)

AND cart contains: Yonex Mavis 350 Shuttlecocks (150,000 VND) + Yonex Astrox 99 Pro (4,500,000 VND)

WHEN customer enters 'SHUTTLE20' and taps Apply

THEN discount = 20% × 150,000 = 30,000 VND (only shuttlecocks eligible)

AND cart total updates: 4,650,000 → 4,620,000 VND

AND voucher appears as a removable tag showing: 'SHUTTLE20 — Save 30,000₫'

[VOUCH-002] Voucher Validation Rules — Must Have

The system shall validate vouchers against ALL of the following conditions. ALL must pass for the voucher to be applied:

#	Validation Rule	Error Message on Failure
V1	Code exists in the system and is active	This voucher code doesn't exist. Please check and try again.
V2	Current date is within valid_from and valid_to range	This voucher expired on {date}. Check 'My Vouchers' for active ones.
V3	Global usage count < usage_limit	This voucher has been fully redeemed and is no longer available.
V4	Per-user usage count < per_user_limit	You've already used this voucher {count}/{limit} times.
V5	Cart subtotal >= min_order_value	Add {remaining} more to use this voucher (minimum order: {min_order_value}).
V6	Cart contains at least one item matching applicable_categories or applicable_product_ids (if scoped)	This voucher only applies to {categories}. Your cart has no matching items.
V7	Customer meets segment conditions (loyalty tier, new customer, etc.) if configured	This voucher is not available for your account type.
V8	No stacking conflict with existing cart discounts (see VOUCH-003)	This voucher can't be combined with your current discount. Remove it first?

Validations are executed in order V1→V8. On first failure, return immediately with the corresponding error message — do not continue to later validations (fail fast). Error messages support i18n (Vietnamese primary, English secondary).

[VOUCH-003] Discount Stacking and Conflict Resolution — Must Have

This is the most complex business rule in the system and the primary integration point between Suggestive Selling and Voucher features.

Scenario: A customer's cart may contain: (a) regular items with item-level promotions (e.g., racket 20% off weekend sale), (b) suggested items that were added via suggestive selling — these may ALSO have their own item-level promotions, and (c) a voucher applied at checkout. The system must resolve all discount interactions correctly.

Rules:

Rule 1: Item-level promotions (automatic discounts) always apply first, on the original price. This includes promotions on suggested items.

Rule 2: Voucher discount applies AFTER item-level promotions, on the post-promotion subtotal.

Rule 3: Only ONE voucher active at a time. No voucher-on-voucher stacking.

Rule 4: If voucher is percentage-based, it calculates on the post-promotion price of eligible items only.

Rule 5: If voucher has a max_discount_amount cap, the discount is capped at that amount regardless of percentage calculation.

Rule 6: CRITICAL — Total combined discount from ALL sources (item promotions + voucher) shall NOT exceed the system-configured max_discount_percentage of the original cart subtotal (default: 50%). If exceeded, the voucher discount is reduced to fit within the cap. The system returns a flag discount_capped: true with explanation.

Acceptance Criteria — Stacking Happy Path:

GIVEN cart: Racket (original 4,500,000, item promo 20% off → pays 3,600,000) + Suggested String (original 200,000, no promo → pays 200,000)

AND customer applies voucher 'SAVE10' (10% off entire cart, no max cap)

WHEN discount is calculated:

Step 1 — Item promotions: Racket discount = 900,000. Post-promo subtotal = 3,800,000

Step 2 — Voucher: 10% of 3,800,000 = 380,000

Step 3 — Total discount: 900,000 + 380,000 = 1,280,000

Step 4 — Cap check: 1,280,000 / 4,700,000 (original subtotal) = 27.2% < 50% cap → OK

THEN customer pays: 4,700,000 - 1,280,000 = 3,420,000 VND

Acceptance Criteria — Cap Exceeded:

GIVEN cart: Racket (original 4,500,000, item promo 40% off → discount 1,800,000) + Suggested String (original 200,000, item promo 30% off → discount 60,000)

AND customer applies voucher 'MEGA20' (20% off entire cart, no max cap)

WHEN discount is calculated:

Step 1 — Item promotions total: 1,860,000. Post-promo subtotal = 2,840,000

Step 2 — Voucher: 20% of 2,840,000 = 568,000

Step 3 — Total discount: 1,860,000 + 568,000 = 2,428,000

Step 4 — Cap check: 2,428,000 / 4,700,000 = 51.6% > 50% cap → EXCEEDS

Step 5 — Reduce voucher: max allowed total = 4,700,000 × 50% = 2,350,000. Voucher reduced to 2,350,000 - 1,860,000 = 490,000 (was 568,000)

THEN customer pays: 4,700,000 - 2,350,000 = 2,350,000 VND

AND UI shows: 'Voucher discount adjusted from 568,000₫ to 490,000₫ due to maximum 50% discount policy'

[VOUCH-004] Remove Voucher — Must Have

Customer can remove an applied voucher by tapping the 'X' on the voucher tag. Upon removal: (a) voucher discount is immediately reversed, (b) cart total recalculates to pre-voucher amount, (c) voucher usage count is NOT incremented (only incremented on successful order placement), (d) a toast confirms: 'Voucher {code} removed'.

[VOUCH-005] Voucher Auto-Invalidation on Cart Change — Must Have

When the cart changes after a voucher has been applied, the system shall re-validate the voucher. If re-validation fails (e.g., item removed causing cart to drop below min_order_value, or all eligible items removed), the voucher is automatically removed with a notification: 'Voucher {code} removed — {reason}'. Specifically:

(a) Customer removes items → cart drops below min_order_value → voucher removed with 'Cart no longer meets minimum {amount}'

(b) Customer removes all items in the voucher's applicable category → voucher removed with 'No eligible items remaining in cart'

(c) Customer removes a suggested item that was the only eligible item for the voucher → same as (b)

5. Data Model

5.1 SuggestiveSelling Module

Entity	Fields	Relationships	Notes
SuggestionRule	id (PK, uuid), name (string), type (enum: product | cart), tier (enum: manual | category | behavioral), priority (int), is_active (bool), valid_from (datetime, nullable), valid_to (datetime, nullable), created_at, updated_at	Has many → SuggestionRuleItem	For type=cart, conditions are stored in CartSuggestionCondition
SuggestionRuleItem	id (PK, uuid), rule_id (FK → SuggestionRule), suggested_product_id (FK → Product via Link), display_order (int), custom_label (string, nullable, e.g., 'Best Match')	Belongs to → SuggestionRule. Links to → Product (via MedusaJS Link Module)	display_order determines rendering sequence within a rule
CartSuggestionCondition	id (PK, uuid), rule_id (FK → SuggestionRule), condition_type (enum: category_missing | threshold_near | brand_match | consumable_upsell), condition_params (jsonb, e.g., {category: 'strings', threshold_pct: 15})	Belongs to → SuggestionRule (where type=cart)	JSON params allow flexible condition config without schema changes
SuggestionEvent	id (PK, uuid), rule_id (FK), source_context (enum: product_view | cart), source_product_id (uuid, nullable), suggested_product_id (uuid), customer_id (uuid), session_id (string), action (enum: impression | tap | add_to_cart | dismiss), created_at	Belongs to → SuggestionRule	Write-heavy, append-only. Consider partitioning by created_at for analytics queries

5.2 VoucherEngine Module (extends Promotion)

Entity	Fields	Relationships	Notes
VoucherConfig	id (PK, uuid, extends Promotion), code (string, unique, indexed), discount_type (enum: percentage | fixed_amount), discount_value (int, in smallest currency unit), min_order_value (int, nullable), max_discount_amount (int, nullable — caps percentage vouchers), applicable_category_ids (uuid[], nullable), applicable_product_ids (uuid[], nullable), stackable_with_promotions (bool, default true), per_user_limit (int, default 1), usage_limit (int, nullable — global), usage_count (int, default 0), user_segment_conditions (jsonb, nullable), valid_from (datetime), valid_to (datetime), is_active (bool), created_at, updated_at	Extends → Promotion. Has many → VoucherUsageLog	code is case-insensitive (stored uppercase). discount_value uses integer arithmetic: 2000 = 20.00% for percentage, or 50000 = 50,000 VND for fixed
VoucherUsageLog	id (PK, uuid), voucher_id (FK → VoucherConfig), customer_id (FK → Customer), order_id (FK → Order),	Belongs to → VoucherConfig, Customer, Order	Created only on successful order placement, NOT

Entity	Fields	Relationships	Notes
	discount_applied (int, actual amount deducted), was_capped (bool), original_discount (int, before cap), applied_at (datetime)		on voucher apply to cart
DiscountCapConfig	id (PK, uuid), max_discount_percentage (int, e.g., 5000 = 50.00%), is_active (bool), updated_at, updated_by (string)	Global singleton config	Managed via admin API. Single active record. History tracked via updated_at

6. API Contracts

6.1 Suggestive Selling APIs

Endpoint	Method	Description	Request / Response Key Fields
GET /store/products/:id/suggestions	GET	Get product-level suggestions for a specific product	Query: store_id (for stock filter). Response: { suggestions: [{ product_id, name, image_url, price, discount_price, label, display_order }] }
GET /store/cart/suggestions	GET	Get cart-level suggestions for the current cart	Query: limit (default 3). Response: { suggestions: [{ product_id, name, image_url, price, rule_id, badge_text }], threshold_info: { target, current, remaining } }
POST /store/suggestions/:id/events	POST	Track a suggestion interaction event	Body: { action: 'impression'|'tap'|'add_to_cart'|'dismiss', source_context, source_product_id, session_id }. Response: 201 Created
POST /admin/suggestion-rules	POST	Create a suggestion rule (admin)	Body: { name, type, tier, items: [{ product_id, display_order, label }], conditions (for cart type) }
PUT /admin/suggestion-rules/:id	PUT	Update a suggestion rule (admin)	Body: partial update fields. Triggers cache invalidation for affected products/carts
DELETE /admin/suggestion-rules/:id	DELETE	Deactivate a suggestion rule (soft delete)	Sets is_active=false. Triggers cache invalidation

6.2 Voucher APIs

Endpoint	Method	Description	Request / Response Key Fields
POST /store/cart/voucher	POST	Apply a voucher to the current cart	Body: { code: 'SHUTTLE20' }. Response: { success, discount_amount, discount_capped, cap_explanation, updated_cart_total, voucher_details: { code, type, value, expires_at } }
DELETE /store/cart/voucher	DELETE	Remove applied voucher from cart	Response: { success, updated_cart_total, message: 'Voucher removed' }
GET /store/customer/vouchers	GET	List vouchers available to the current customer	Response: { vouchers: [{ code, description, discount_type, discount_value, valid_to, min_order, applicable_categories }] }
POST /admin/vouchers	POST	Create a new voucher (admin)	Body: full VoucherConfig fields. Response: created voucher with generated code if not provided
GET /admin/vouchers/:id/analytics	GET	Voucher usage analytics (admin)	Response: { total_uses, total_discount_given, avg_order_value, capped_count, conversion_rate }

7. MedusaJS Workflow Specifications

All multi-step operations are implemented as MedusaJS Workflows with compensation (rollback) support. Each step is independently testable and the workflow can resume or rollback from any failure point.

7.1 evaluateSuggestions Workflow

Triggered by: product detail page load (product-level) or cart.updated event (cart-level).

Step	Action	Compensation	Output
1. resolveContext	Determine source type (product or cart), load source data, load customer profile	— (read-only)	{ source_type, source_data, customer }
2. loadActiveRules	Query SuggestionRule where type matches, is_active=true, within valid date range, ordered by priority	— (read-only)	{ rules[] }
3. evaluateRules	For each rule, evaluate conditions (Tier 1 → 2 → 3). Collect candidate products.	— (read-only)	{ candidates[] }
4. filterCandidates	Apply exclusion filters: already in cart, out of stock, dismissed this session, purchased in last 30 days	— (read-only)	{ filtered_candidates[] }
5. rankAndLimit	Sort by tier priority then display_order. Limit to max results (5 for product, 3 for cart)	— (read-only)	{ final_suggestions[] }
6. enrichWithPricing	Load current prices, discount prices, stock status for each suggestion	— (read-only)	{ enriched_suggestions[] }
7. cacheResults	Store in Redis: product:{id}:suggestions or cart:{id}:suggestions with TTL 5min	Delete cache key	{ cache_key }

7.2 applyVoucher Workflow

Triggered by: POST /store/cart/voucher API call.

Step	Action	Compensation	Output
1. normalizeCode	Uppercase the code, trim whitespace	— (pure function)	{ normalized_code }
2. lookupVoucher	Query VoucherConfig by code. Return 404 if not found	— (read-only)	{ voucher_config }
3. validateExpiry	Check valid_from <= now <= valid_to	— (read-only)	{ is_valid, error_msg }
4. validateUsage	Check global usage_count < usage_limit AND per-user count < per_user_limit (atomic Redis check)	— (read-only)	{ is_valid, error_msg }
5. validateCart	Check min_order_value, applicable categories/products in cart	— (read-only)	{ eligible_items[], is_valid, error_msg }

Step	Action	Compensation	Output
6. validateSegment	Check customer segment conditions (if configured)	— (read-only)	{ is_valid, error_msg }
7. calculateDiscount	Compute raw discount: percentage of eligible items or fixed amount. Apply max_discount_amount cap if set	— (calculation)	{ raw_discount, voucher_capped }
8. enforceGlobalCap	Sum all item-level promotions + voucher discount. Check against DiscountCapConfig.max_discount_percentage. Reduce voucher if exceeds.	— (calculation)	{ final_discount, is_globally_capped, explanation }
9. attachToCart	Set voucher on cart entity, update cart totals with final discount amount	Remove voucher from cart, revert totals	{ updated_cart }

7.3 revalidateVoucherOnCartChange Workflow

Triggered by: cart.updated subscriber (after item add/remove/quantity change).

Step	Action	Compensation	Output
1. checkVoucherExists	Check if cart has an active voucher. If not, exit early.	— (read-only)	{ has_voucher, voucher_config }
2. revalidateCart	Re-run steps 5-6 of applyVoucher (cart validation, segment check) against updated cart	— (read-only)	{ still_valid, failure_reason }
3a. (if valid) recalculate	Re-run steps 7-8 (discount calculation + global cap) with updated cart totals	Revert to previous totals	{ new_discount }
3b. (if invalid) removeVoucher	Remove voucher from cart, revert to pre-voucher totals	Re-attach voucher (unlikely needed)	{ removed, reason_message }
4. notifyFrontend	Push updated cart state to frontend. If removed, include toast message with reason	— (fire-and-forget)	{ notification_sent }

8. Edge Cases & Business Rules Matrix

These edge cases represent the intersection of Suggestive Selling and Voucher features — the scenarios where both systems interact and complexity is highest.

ID	Scenario	Expected Behavior	Priority
EC-01	Suggested items added to carts have its own 30% item promo. Customers also has 20% voucher. Combined discount approaches 50% cap.	Calculate: item promos first → voucher on post-promo total → check global cap. If it exceeds 50%, reduce the voucher amount only (never reduce item promos). Show cap explanation to customer.	Must
EC-02	Customer applies voucher scoped to 'Strings' category. Then removes all strings from cart (including one added via suggestion).	cart.updated triggers revalidateVoucherOnCartChange. Step 2 fails (no eligible items). Voucher auto-removed with: 'Voucher {code} removed — no eligible items remaining in cart.'	Must
EC-03	Voucher gives 50% off. Suggested item has 50% item promo. Combined = 100% → cart total would be 0 or negative.	Global cap (50%) prevents this. Voucher reduced to stay within cap. Cart total always > 0 (minimum 1 VND after all discounts). System logs a warning for admin review.	Must
EC-04	Two concurrent requests: one applies voucher, other removes the last eligible item from cart.	Optimistic locking on cart entity. Second request triggers revalidation. If voucher is no longer valid, remove it. Both operations succeed atomically — no inconsistent state.	Must
EC-05	Customer views suggestions on product page, adds suggested item, navigates to cart. Cart suggestions should NOT re-suggest the just-added item.	evaluateSuggestions workflow Step 4 (filterCandidates) checks current cart items. Recently added suggested item is excluded. Suggestion cache invalidated by cart.updated event.	Must
EC-06	Voucher has per_user_limit=1. Customer applies voucher, removes it, tries to apply again in same session.	Usage count is only incremented on ORDER PLACEMENT, not on apply-to-cart. So re-applying in same session is allowed. This is by design — prevents punishing customers who are exploring.	Must
EC-07	Suggested product goes out of stock between suggestion render and customer tapping 'Add to Cart'.	addToCart API re-checks stock at execution time (not cache). Returns 409 with: '{Product} just went out of stock. We've updated your suggestions.' Frontend refreshes suggestion section.	Must
EC-08	Cart has 3 items. Voucher applied successfully. Customer adds a suggested item that triggers a price recalculation pushing total past a new promotion tier (e.g., 'Spend 5M get extra 5% off').	New tier promotion is applied as item-level discount. Then voucher is recalculated on new post-promo subtotal. Global cap re-checked. Customer benefits from cascading discounts up to the cap.	Should
EC-09	Admin deactivates a suggestion rule while customers have cached suggestions showing products from that rule.	Cache TTL (5min) handles eventual consistency. Worst case: customer sees a stale suggestion for up to 5 minutes. Adding the suggested product to cart still works (product exists, just the suggestion rule is inactive). No error for customer.	Should
EC-10	Rapid voucher code attempts (brute force) — trying random codes to find valid ones.	Rate limit: max 5 failed voucher attempts per customer per 15-minute window. After 5 failures, return 429: 'Too many attempts. Please try again in	Must

ID	Scenario	Expected Behavior	Priority
		{minutes} minutes.' Log IP + customer_id for security monitoring.	

9. Non-Functional Requirements

9.1 Performance

Metric	Target	Notes
Product-level suggestion load	< 800ms (p95)	Cached in Redis. Cold miss falls back to DB query < 500ms + cache write
Cart-level suggestion evaluation	< 600ms (p95)	Triggered async by cart.updated subscriber. Frontend shows skeleton loader
Voucher validation (apply)	< 400ms (p95)	Redis-based usage check. DB query only for voucher config lookup
Cart total recalculation (after voucher/discount change)	< 300ms (p95)	All arithmetic in application layer, no DB writes during calculation
Suggestion cache hit rate	> 85%	5-min TTL, invalidated on cart change. Monitor via Redis metrics

9.2 Security

SEC-01: All discount calculations are server-side only. Frontend displays are informational; actual prices come from the cart API response. Tampering with frontend amounts has no effect on actual charges.

SEC-02: Voucher code brute-force protection — 5 failed attempts per customer per 15-minute window → 30-minute cooldown. Logged for monitoring.

SEC-03: Voucher codes are case-insensitive, stored uppercase, minimum 6 characters, alphanumeric only — reduces guessing surface.

SEC-04: Admin APIs for rule/voucher management require authentication + admin role. Customer-facing APIs are scoped to the authenticated customer's data only.

9.3 Data Integrity

INT-01: All monetary values stored as integers in smallest currency unit (VND has no decimal, so 1 = 1 VND). No floating-point arithmetic anywhere in discount calculations.

INT-02: Voucher usage_count is incremented atomically (Redis INCR or DB UPDATE with WHERE clause) to prevent race conditions.

INT-03: Cart total is the authoritative source of pricing truth. It is recalculated from scratch (not incrementally) on every change to prevent drift.

INT-04: VoucherUsageLog is append-only and immutable after creation. Provides audit trail for all voucher redemptions.

10. Acceptance Test Checklist

This checklist is organized by feature and maps directly to requirements for automated test implementation.

10.1 Suggestive Selling Tests

Test ID	Scenario	Validates	Type
T-SUGG-01	Product with 3 manual suggestions → all 3 displayed in order	SUGG-001 Tier 1	Integration
T-SUGG-02	Product with 1 manual suggestion → backfill with 2-4 category complements	SUGG-001 Tier 2	Integration
T-SUGG-03	Suggested product already in cart → excluded from suggestions	SUGG-002 (a)	Unit
T-SUGG-04	Suggested product out of stock → excluded	SUGG-002 (b)	Unit
T-SUGG-05	Suggestion dismissed → not shown again in session	SUGG-002 (c)	Integration
T-SUGG-06	Add suggested product via one-tap → item in cart, toast shown	SUGG-003	E2E
T-SUGG-07	Cart with racket, no string → CR-01 fires, suggests strings	SUGG-004 CR-01	Unit
T-SUGG-08	Cart near free shipping threshold → badge shown on affordable suggestion	SUGG-004 CR-02	Unit
T-SUGG-09	Cart change → suggestions refresh (old cache invalidated)	SUGG-005	Integration
T-SUGG-10	Suggestion events tracked (impression, tap, add, dismiss)	SUGG-006	Integration

10.2 Voucher Tests

Test ID	Scenario	Validates	Type
T-VOUCH-01	Valid voucher applied → discount shown, total updated	VOUCH-001	Integration
T-VOUCH-02	Invalid code → specific error message	VOUCH-002 V1	Unit
T-VOUCH-03	Expired voucher → expiry error with date	VOUCH-002 V2	Unit
T-VOUCH-04	Per-user limit exceeded → usage count error	VOUCH-002 V4	Unit
T-VOUCH-05	Cart below min_order → amount needed shown	VOUCH-002 V5	Unit
T-VOUCH-06	No eligible items in cart → category error	VOUCH-002 V6	Unit
T-VOUCH-07	Item promo 20% + voucher 10% → both apply, under cap	VOUCH-003 happy path	Unit
T-VOUCH-08	Item promo 40% + voucher 20% → voucher reduced by cap	VOUCH-003 cap exceeded	Unit

Test ID	Scenario	Validates	Type
T-VOUCH-09	Suggested item with 50% promo + voucher 50% → cap prevents negative total	EC-03	Unit
T-VOUCH-10	Remove voucher → totals reverted, no usage increment	VOUCH-004	Integration
T-VOUCH-11	Remove eligible items after voucher applied → voucher auto-removed	VOUCH-005	Integration
T-VOUCH-12	5 failed attempts → rate limited	EC-10	Integration

— End of Document —

SRS v1.0 — Suggestive Selling + Voucher at Checkout — Generated with AI-Assisted Requirements Engineering